

FIT3155

Advanced algorithms and data structures

Week 4 lecture notes by Taylor Kearney

Based on lecture slides by Arun Konagurthu



Faculty of Information Technology

September 11, 2025

Contents

1	Suffix trees	2
2	Implicit suffix trees	7
3	Ukkonen's algorithm for linear time construction of suffix trees	8
3.1	Overview	8
3.2	Naive implicit suffix tree construction	9
3.3	Speeding up traversal with suffix links	16
3.4	Skip-counting	22
3.5	Space efficient representation of edge labels	24
3.6	Extension termination criterion (the showstopper rule)	25
3.7	Rapid leaf extension	26
3.8	Rule 3s require a little more than nothing	28
3.9	Summary	29
3.10	Extended example	32

1 Suffix trees

To this point we have only examined linear time solutions to the exact pattern matching problem that preprocess the pattern. However, there are also linear time solutions that preprocess the text. Although we will find the worst case complexity of these methods to be the same (or similar depending on the level of granularity of our analysis), whether it is best to preprocess the pattern or the text is often dictated by the problem at hand. In fact, often we are not given a choice as in many problems only one of the pattern or the text are made available for preprocessing.

Definition 1: The substring matching problem

Given a reference text $\mathbf{txt}[1 \dots n]$, preprocess $\mathbf{txt}$ such that any given $\mathbf{pat}[1 \dots m]$ can be searched in time proportional to the length of the pattern, $O(m)$.

In the substring matching problem we are given access to the text and need to preprocess it in order to support rapid queries. In contrast, the exact pattern matching problem gave us a pattern and asked us to find it in an initially unknown text. The algorithms that we have considered previously enable us to efficiently search for a single pattern within multiple large texts. However, now we are tasked with efficiently searching for multiple patterns within a single text. This reversal of roles of the pattern and the text reduces the effectiveness of the methods that preprocess the pattern and to remedy this we need to construct methods that preprocess the text.

There are several data structures that can be constructed from the text that support pattern matching in $O(m)$ -time. For instance, we examined suffix trees and the burrows wheeler transform (BWT) in FIT2004. Here we choose to focus on suffix trees as they are tremendously versatile data structures that enable efficient solutions to a multitude of important problems (see tutorial 3 and assignment 2 for examples). However, naive construction of suffix trees requires $O(n^2)$ -time and yields an $O(n^2 + km)$ -time solution to the substring matching problem; assuming we are given k patterns of the same length. On the surface they do not appear to be an attractive alternative to exact pattern matching algorithms that promise an $O(m)$ -time preprocessing phase and an $O(n)$ -time search phase for each pattern, giving $O(km + kn)$ overall. Fortunately, there exists an ingenious algorithm for constructing a suffix tree in linear time known as *Ukkonen's algorithm*, and with it suffix trees allow us to solve the substring matching problem in $O(n + km)$ -time. But before we examine the inner workings of Ukkonen's, we will quickly remind ourselves of what a suffix tree is and examine their connection to the *implicit suffix trees*, which are central to the Ukkonen's algorithm.

Definition 2: Suffix trees

A suffix tree for a string $\mathbf{str}[1 \dots n]$ is a tree with the following properties.

1. It has exactly n leaves numbered 1 through n .
2. Except for the root, every internal node has at least two children.
3. Each edge is labelled with a non-empty substring of $\mathbf{str}$.
4. Every node has at most one edge beginning with any given character of the alphabet.
5. The string-label of the path starting at the root and ending at the leaf numbered i is the suffix $\mathbf{str}[i \dots n]$, for $i = 1 \dots n$.

Recall that we can also define suffix trees as path compressed suffix tries.

Example 1: Suffix tree for abcabz

Let's look at a simple example, $\text{str}[1 \dots 6] = \text{a}^1 \text{b}^2 \text{c}^3 \text{a}^4 \text{b}^5 \text{z}^6$ whose suffix tree is shown below,

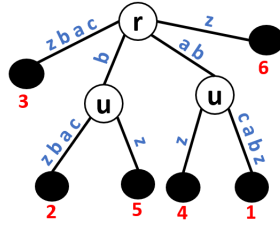


Figure 1: The suffix tree for $\text{str}[1 \dots 6] = \text{abcabz}$

We can see that this tree has all 5 properties we require of a suffix tree. Specifically, it has:

- 6 leaves numbered 1 through 6,
- every internal node has at least two children except the root which has four,
- each edge is labelled with a substring of str ,
- at every node no two edges start with the same character,
- any path from the root to a leaf node will yield a suffix of str .

Example 2: Implicit suffix tree for abcab

On the surface our definition of a suffix tree seems sound, but what happens if we remove the last character from our string? We would obtain $\text{str}[1 \dots 5] = \text{a}^1 \text{b}^2 \text{c}^3 \text{a}^4 \text{b}^5$ and the corresponding tree would be,

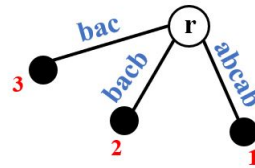


Figure 2: The implicit suffix tree for $\text{str}[1 \dots 5] = \text{abcab}$

But this isn't a suffix tree! In particular, it doesn't satisfy rules 1 and 5. The issue is that the suffixes $\text{str}[4 \dots 5] = \text{'ab'}$ and $\text{str}[5] = \text{'b'}$ are prefixes of the suffixes $\text{str}[1 \dots 5] = \text{'abcab'}$ and $\text{str}[2 \dots 5] = \text{'bcab'}$ respectively. In contrast, the string $\text{str}[1 \dots 6] = \text{'abcabz'}$ has no such suffixes as the last character ('z') appears nowhere else in the string.

Example 2 demonstrates that not all strings have an associated suffix tree, but we can circumvent this issue by adding a special *terminal character* to the end of our string. In this unit (unless stated otherwise) we will take this character to be '\$'.

Definition 3: Terminal character

The terminal character ‘\$’ is defined to be a character in the alphabet such that,

1. It occurs only once in the string and is always the last character in the string.
2. It is lexicographically **smaller** than any other character in the alphabet.

Example 3: Suffix tree for abcab

To construct a valid suffix tree for $\text{str}[1 \dots 5] = \text{abcab}$ we can append ‘\$’ to give $\text{str}[1 \dots 6] = \text{abcab\$}$, which then yields the tree,

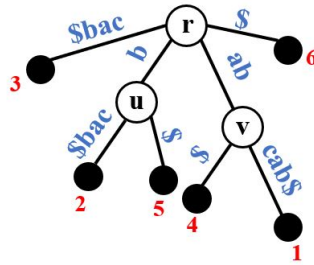


Figure 3: The suffix tree for $\text{str}[1 \dots 6] = \text{abcab\$}$

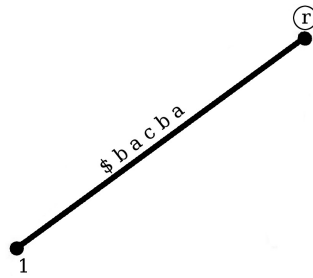
You should check that this tree satisfies all the required properties.

Strictly speaking the suffix tree given in example 3 is the suffix tree for ‘abcab\$’ and not ‘abcab’, but as the latter doesn’t have a corresponding suffix tree, and the former has all the practical and conceptual properties that one might want in a suffix tree for ‘abcab’, we will treat the two as synonymous from this point forward. As such, anytime we refer to the suffix tree for a string $\text{str}[1 \dots n]$ we are actually referring to the suffix tree of $\text{str}[1 \dots n]\$$.

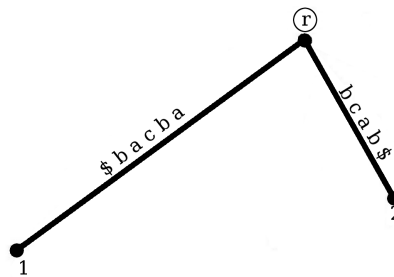
Naive suffix tree construction

For the sake of completeness we will briefly examine the naive construction of suffix trees. This is assumed knowledge and you should revise your notes from previous units for full discussion of the naive algorithm as we will only provide a refresher through way of example here. The suffix tree in example 3 can be constructed naively as follows:

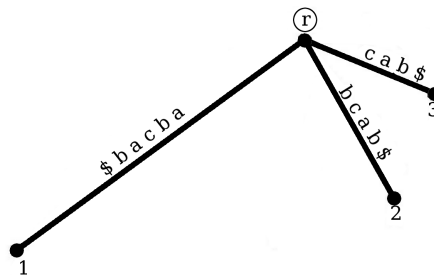
- Start with the (empty) root node of the suffix tree.
- Insert suffix **1** ($\text{str}[1\dots]$) into an empty tree.
- Call the resultant tree T_1 .



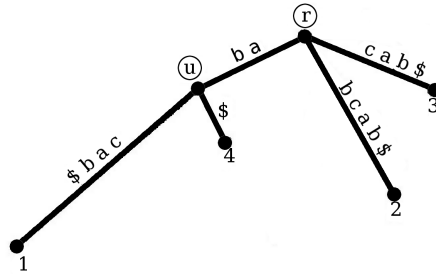
- Insert suffix **2** into T_1 .
- Note $\mathbf{str}[2\dots]$ is not a prefix of $\mathbf{str}[1\dots]$
- So create a new leaf node for $\mathbf{str}[2\dots]$ suffix, branching off at the root.
- Call resultant tree T_2



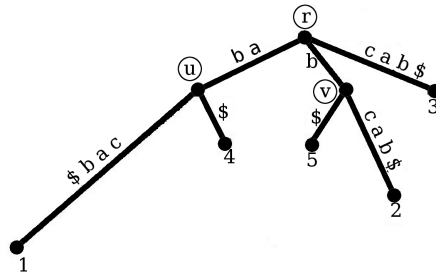
- Insert suffix **3** into T_2 .
- Note $\mathbf{str}[3\dots]$ is not a prefix of $\mathbf{str}[1\dots]$ or $\mathbf{str}[2\dots]$.
- So create a new leaf node for $\mathbf{str}[3\dots]$ suffix, again branching off at the root.
- Call resultant tree T_3



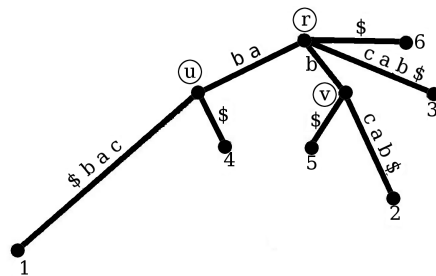
- Inserting suffix **4** into T_3 .
- Note $\mathbf{str}[4..5] = ab$ is the longest common prefix shared with the suffix $\mathbf{str}[1\dots]$.
- So, a new node (u) is inserted ...
- ... along the edge between the root and the leaf node 1...
- ... with another edge branching off u to the new leaf node 4
- Call resultant tree T_4



- Insert suffix 5 into T_4 .
- Note $\text{str}[5..5] = b$ is the longest common prefix shared with suffix $\text{str}[2..]$.
- So, a new node (v) is inserted...
- ... along the edge between the root and the leaf node 2...
- ... with another edge branching out from v to the new leaf node 5
- Call resultant tree T_5



- Insert suffix 6 into T_5 .
- Note the suffix $\text{str}[6..6]$ denotes the special terminal character \$.
- This creates a new isolated edge branching off the root.



Question 1

Assuming as input a string, $\text{str}[1 \dots n]$, show that the time complexity of the naive method presented above is $O(n^2)$.

Space efficient representation of suffix trees

Explicitly storing the substring labels of each edge in our suffix tree is convenient conceptually, but requires $O(n^2)$ space for a string of length n . Fortunately, we don't need to store the edge labels explicitly. Instead we can store them **implicitly** using a tuple (j, i) to denote the substring $\text{str}[j \dots i]$, where $1 \leq (j \leq i) \leq n$. This reduces the space required to store the suffix tree to $O(n)$ (assuming a fixed size alphabet).

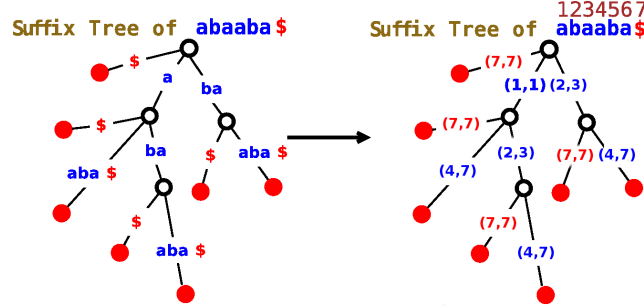


Figure 4: By implicitly storing the edge labels using the start and end indices of the corresponding substrings we can store a suffix tree in $O(n)$ space.

Question 2

Prove that the space efficient representation of suffix trees requires $O(n)$ space. You may assume the size of the alphabet under consideration is fixed.

2 Implicit suffix trees

We have now established a robust definition for suffix trees, but this does not make the trees in example 1 and 2 insignificant. These are in fact the *implicit suffix trees* for the strings 'abcabz' and 'abcab' respectively.

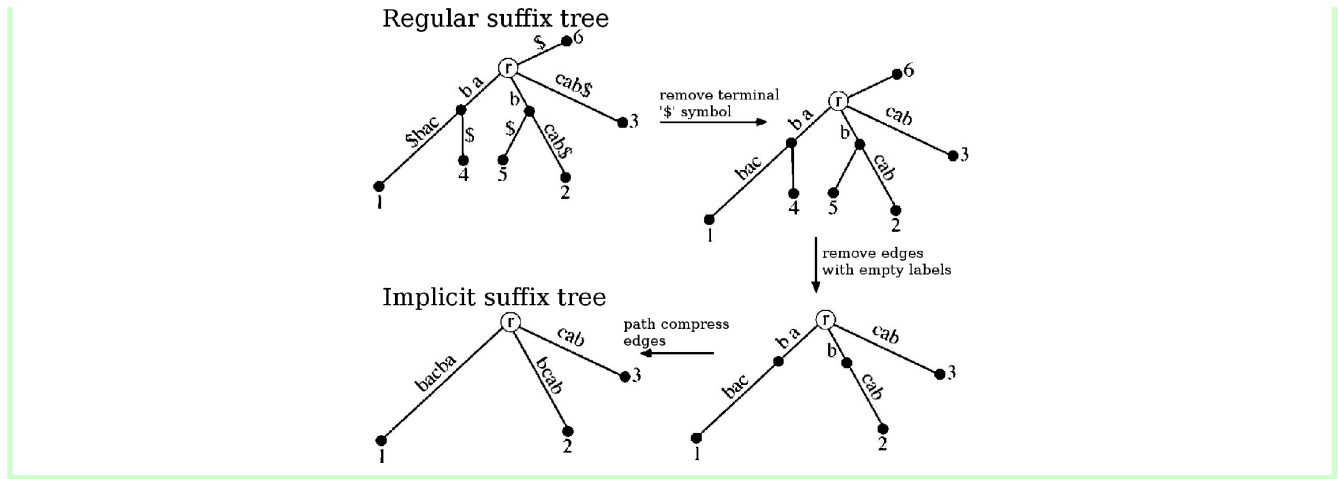
Definition 4: Implicit suffix trees

The implicit suffix tree for a string $\text{str}[1 \dots n]$ is the tree obtained from the suffix tree for $\text{str}[1 \dots n]$ by,

1. Removing all the terminal characters ('\$') from the tree.
2. Deleting all the edges labelled with empty substrings in the resultant tree.
3. Removing all internal nodes that don't have at least two children.

Example 4: Constructing an implicit suffix tree from a suffix tree.

Let's examine the process of constructing an implicit suffix tree when given a regular suffix tree for,
 $\text{str}[1 \dots n] = \overset{1}{a} \overset{2}{b} \overset{3}{c} \overset{4}{a} \overset{5}{b}$.



Notice that the tree in example 4 only contains the suffixes $\text{str}[5 \dots 6] = \text{'ab'}$ and $\text{str}[6] = \text{'b'}$ implicitly hence the name **implicit** suffix tree. Furthermore, we can see that the final tree is identical to the tree we found in example 2. This suggests an alternate definition for implicit suffix trees.

Definition 5: Implicit suffix trees (alternate definition)

An implicit suffix tree for the string $\text{str}[1 \dots n]$ is the tree obtained when attempting to build the suffix tree for str **without** first appending the terminal character ($\text{'$'}$) to str .

These definitions imply that we can obtain the regular suffix tree that corresponds to an implicit suffix tree by simply appending $\text{'$'}$ to all the suffixes contained within the tree. Although we are yet to describe how to do this algorithmically this idea is at the heart of Ukkonen's algorithm.

3 Ukkonen's algorithm for linear time construction of suffix trees

Recall that our goal is to develop a method for building the suffix tree that corresponds to a string $\text{str}[1 \dots n]$ in $O(n)$ -time. We have already seen that naive construction requires $O(n^2)$ -time, but we have just discovered an alternate method for constructing suffix trees.

First construct the implicit suffix tree for $\text{str}[1 \dots n]$, and then extend every suffix in this tree by $\text{'$'}$ to obtain the corresponding regular suffix tree. This method always yields the correct regular suffix tree for $\text{str}[1 \dots n]$ because the terminal character only appears at the end of the string by definition, and when the last character in a string is found nowhere else in the string the implicit suffix tree for the string is the same as the regular suffix tree (see example 1). That is, the implicit suffix tree for $\text{str}[1 \dots n]\text{'$'}$ is the regular suffix tree for $\text{str}[1 \dots n]\text{'$'}$. However, this viewpoint can be restrictive as it tempts us to construct the required implicit suffix tree using the naive method for regular suffix tree construction, which obviously will not yield any improvement on the worst case time complexity. To achieve the desired linear bound we need to invert our thinking and grow the suffixes in our tree incrementally instead of successively inserting the complete suffixes into our tree.

3.1 Overview

At a high level Ukkonen's algorithm is relatively simple, and although the 'devil is in the detail,' we will abstract ourselves from these complications initially to establish the algorithm's overall structure. As alluded to in the previous section, the goal of Ukkonen's algorithm is to construct the regular suffix tree for $\text{str}[1 \dots n]$ by first constructing the corresponding implicit suffix tree. The algorithm proceeds over n *phases* and in each phase $i + 1$ (where $1 \leq i + 1 \leq n$) the implicit suffix tree $\mathcal{I}_{i+1}$ for the **prefix** $\text{str}[1 \dots i + 1]$ is constructed. Specifically in each phase $i + 1$, $\mathcal{I}_{i+1}$ is computed incrementally using $\mathcal{I}_i$, the implicit suffix tree constructed in the previous phase. At the conclusion of the n phases we have constructed $\mathcal{I}_n$ - the implicit suffix tree for $\text{str}[1 \dots n]$ - which can be extended to the regular suffix tree for $\text{str}[1 \dots n]$ by appending the terminal character $\text{'$'}$ and running one last iteration of the algorithm¹.

¹When implementing Ukkonen's algorithm it is easiest to append the terminal character to $\text{str}[1 \dots n]$ at the beginning of the algorithm and then perform $n + 1$ phases to obtain the regular suffix tree for $\text{str}[1 \dots n]$.

By definition $\mathcal{I}_i$ contains (implicitly) all the suffixes of $\mathbf{str}[1 \dots i]$ and in phase $i + 1$ these suffixes all undergo *suffix extensions* (often just called extensions) to accommodate the new character $\mathbf{str}[i + 1]$. Once every suffix of $\mathbf{str}[1 \dots i]$ has been extended by $\mathbf{str}[i + 1]$, and the new suffix $\mathbf{str}[i + 1]$ has been added to the tree (equivalent to extending the empty suffix by $\mathbf{str}[i + 1]$), the tree will contain all the suffixes of $\mathbf{str}[1 \dots i + 1]$ and we obtain $\mathcal{I}_{i+1}$. Within phase $i + 1$ extensions proceed as follows: for any suffix of $\mathbf{str}[1 \dots i]$ starting at position j , where $1 \leq j \leq i + 1$,

- find the end of the path from the root node in the current state of the implicit suffix tree that corresponds to the suffix $\mathbf{str}[j \dots i]$.
- Extend this path by appending $\mathbf{str}[i + 1]$ to the suffix so that the tree now contains a path from the root that corresponds to the suffix $\mathbf{str}[j \dots i + 1]$.

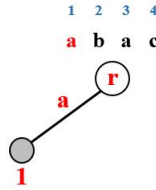
3.2 Naive implicit suffix tree construction

There are three types of extensions that occur once the path $\mathbf{str}[j \dots i]$ has been found and by detailing these we can transform the previous overview into a naive method for constructing implicit suffix trees. We can then derive Ukkonen's algorithm by making several intuitive improvements that will eventually yield an $O(n)$ -time method. To proceed we'll consider building the implicit suffix tree for $\mathbf{str}[1 \dots 4] = abac$ and generalise the extensions that occur as we come to them.

To begin we must construct $\mathcal{I}_1$ from an empty tree. Recall that $\mathcal{I}_1$ contains all suffixes of the prefix $\mathbf{str}[1]$, which is just a single character $\mathbf{str}[1] = a$. Thus, we only have one extension to make as $1 \leq j \leq i + 1 \implies j = 1$.

Phase: $i + 1 = 1$, extension $j = 1$

To insert $\mathbf{str}[1] = a$ we create a new root node $\textcircled{r}$ and a new leaf numbered 1. We then assign the character a as the edge label between the new node $\textcircled{r}$ and leaf 1.

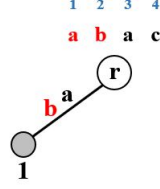


Shortly we will see that this extension can be thought of as what we will call a *rule 2* extension, but for now we will treat it as the base case of the algorithm.

Now that all extensions in phase $i + 1 = 1$ have been made we proceed to phase $i + 1 = 2$, where we need to construct $\mathcal{I}_2$ from $\mathcal{I}_1$. In particular, we need to extend the path $\mathbf{str}[1] = a$ in $\mathcal{I}_1$ to $\mathbf{str}[1 \dots 2] = ab$ and insert $\mathbf{str}[2] = b$, the remaining suffix of the prefix $\mathbf{str}[1 \dots 2]$. This gives us two extensions overall in phase 2 and $1 \leq j \leq 2$.

Phase: $i + 1 = 2$, extension $j = 1$

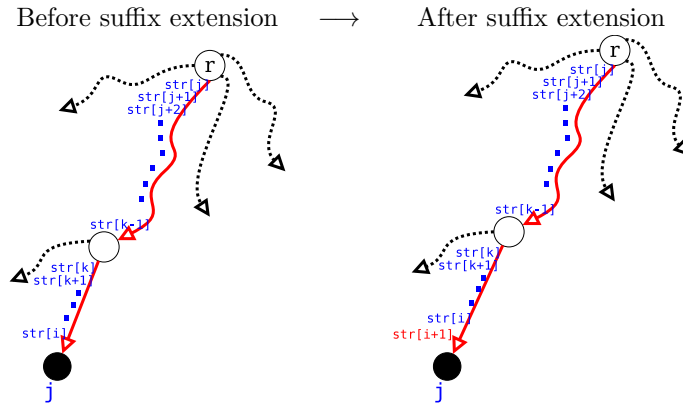
To insert $\mathbf{str}[1 \dots 2] = ab$ we first find the path $\mathbf{str}[1] = a$. As this path ends at the leaf 1 we simply adjust the **label of the edge** to account for the new character $\mathbf{str}[2] = b$, so that the edge is now labeled by $\mathbf{str}[1 \dots 2] = ab$.



The previous extension is the first of three types that can occur when constructing implicit suffix trees in this manner and we define an *extension rule* that describes how to perform this kind of extension.

Definition 6: Rule 1 extensions

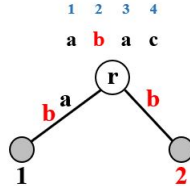
When performing extension j in phase $i + 1$ first find the path $\mathbf{str}[j \dots i]$ in $\mathcal{I}_i$. If this path **ends at a leaf**, adjust the **label of the edge** to that leaf to account for the added character $\mathbf{str}[i + 1]$.



Having completed the first extension in the phase we proceed immediately to the next.

Phase: $i + 1 = 2$, extension $j = 2$

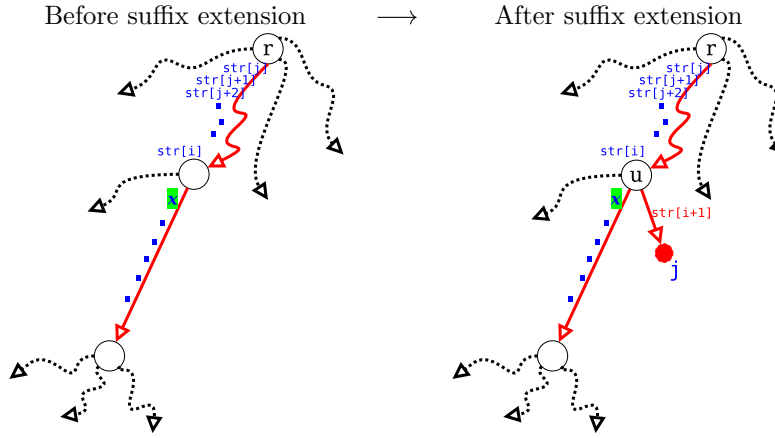
To insert $\mathbf{str}[2] = b$ we first note that the path ($\mathbf{str}[j \dots i] = \mathbf{str}[2 \dots 1]$) that needs to be extended is the empty string, and so our extension is made directly from $\textcircled{r}$. There is no path from the root that starts with $\mathbf{str}[2] = b$ we must create a new leaf numbered 2 and assign the character b as the edge label between the $\textcircled{r}$ and leaf 2.



The $j = 2$ extension is clearly not a rule 1 extension as our path does not end at a leaf, and is in fact an example of a *rule 2* extension defined as follows.

Definition 7: Rule 2 extensions (case 1)

When performing extension j in phase $i + 1$ first find the path $\mathbf{str}[j \dots i]$ in $\mathcal{I}_i$. If this path **does not** end at a leaf, and the next character in the existing path is some $x \neq \mathbf{str}[i + 1]$, and $\mathbf{str}[i]$ and x are separated by an existing node (u) , then create a new leaf numbered j ; assign character $\mathbf{str}[i + 1]$ as the edge label between the (u) and the leaf j .

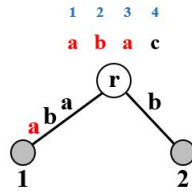


In extension $j = 2$ in phase $i + 1 = 2$ the path $\mathbf{str}[2 \dots 1]$ was the empty string and so ended at the root. The next character in the only existing path was $\mathbf{str}[1] = a \neq \mathbf{str}[2] = b$ and so we created a new leaf numbered 2 and labeled the edge between leaf 2 and the root with $\mathbf{str}[2] = b$. The base case is also essentially a rule 2 extension of this kind as we begin with a singleton node in the root and our initial path is $\mathbf{str}[1 \dots 0]$, which is the empty string. As there are no non-empty paths from the root in phase $i + 1 = 1$ we cannot find any path continuing with the character $\mathbf{str}[1]$ and so we must create a new leaf numbered 1 and label the edge between it and the root with $\mathbf{str}[1]$.

With both extensions in phase $i + 1 = 2$ complete we now move onto phase $i + 1 = 3$ where we must make three extensions, one each for $1 \leq j \leq 3$.

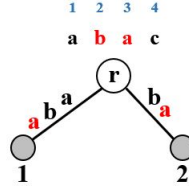
Phase: $i + 1 = 3$, extension $j = 1$

To insert $\mathbf{str}[1 \dots 3] = aba$ we first follow the path $\mathbf{str}[j \dots i] = \mathbf{str}[1 \dots 2] = ab$ from (r) . This path ends at leaf 1 so we make the extension using rule 1 and extend the edge label between (r) and leaf 1 by $\mathbf{str}[3] = a$.

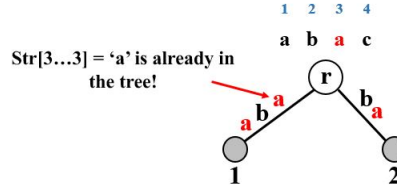


Phase: $i + 1 = 3$, extension $j = 2$

To insert $\text{str}[2 \dots 3] = ba$ we first follow the path $\text{str}[j \dots i] = \text{str}[2 \dots 2] = b$ from $\textcircled{r}$. This path ends at leaf 2 so we make the extension using rule 1 and extend the edge label between $\textcircled{r}$ and leaf 2 by $\text{str}[3] = a$.


Phase: $i + 1 = 3$, extension $j = 3$

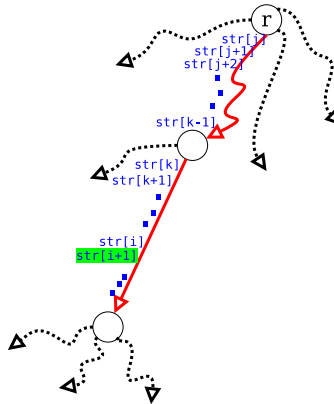
To insert $\text{str}[3] = a$ we first follow the path $\text{str}[j \dots i] = \text{str}[3 \dots 2]$, which is the empty string. Starting from $\textcircled{r}$ we look for a path continued by $\text{str}[3] = a$ and we find one. This means that the suffix $\text{str}[3] = a$ is already in the tree (implicitly) and we need to do nothing.



In the previous extension we found that the suffix $\text{str}[3]$ of the prefix $\text{str}[1 \dots 3]$ that we were trying to insert was already contained (implicitly) in our tree. This kind of extension is the third type that can occur and is called a *rule 3* extension defined as follows.

Definition 8: Rule 3 extensions

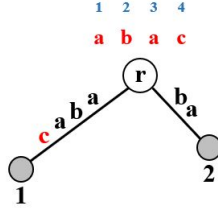
When performing extension j in phase $i + 1$ first find the path $\text{str}[j \dots i]$ in $\mathcal{T}_i$. If this path **does not** end at a leaf, and the next character in that path is $\text{str}[i + 1]$, then $\text{str}[i + 1]$ is already in the tree, and no further action is needed.



After we make extension $j = 3$ in phase $i + 1 = 3$ we are ready to move onto phase $i + 1 = 4$. In this final phase we need to make four extensions, one each for $1 \leq j \leq 4$.

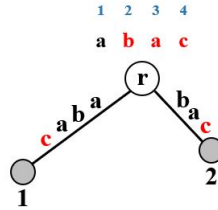
Phase: $i + 1 = 4$, extension $j = 1$

To insert $\mathbf{str}[1 \dots 4] = abac$ we first follow the path $\mathbf{str}[j \dots i] = \mathbf{str}[1 \dots 3] = aba$ from (r) . This path ends at leaf 1 so we make the extension using rule 1 and extend the edge label between (r) and leaf 1 by $\mathbf{str}[4] = c$.



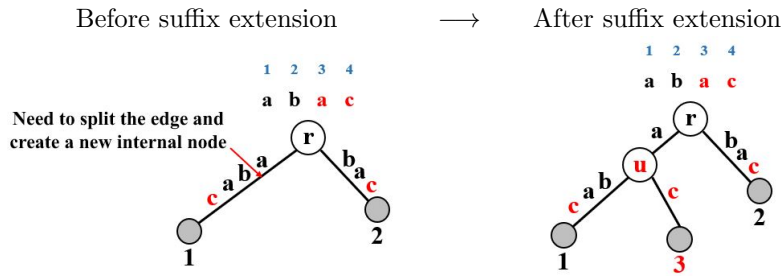
Phase: $i + 1 = 4$, extension $j = 2$

To insert $\mathbf{str}[2 \dots 4] = bac$ we first follow the path $\mathbf{str}[j \dots i] = \mathbf{str}[2 \dots 3] = ba$ from (r) . This path ends at leaf 2 so we make the extension using rule 1 and extend the edge label between (r) and leaf 2 by $\mathbf{str}[4] = c$.



Phase: $i + 1 = 4$, extension $j = 3$

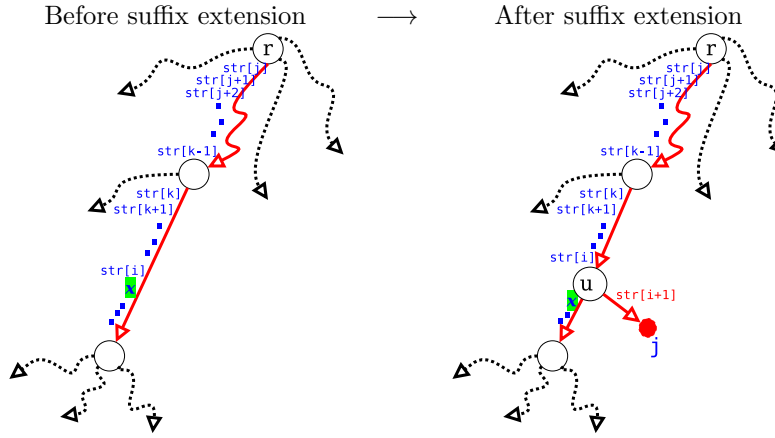
To insert $\mathbf{str}[3 \dots 4] = ac$ we first follow the path $\mathbf{str}[j \dots i] = \mathbf{str}[3 \dots 3] = a$ from (r) . This path ends part way along the edge between the root and leaf 1, but it continues with the character b that does not match $\mathbf{str}[4] = c$. Therefore we must split the edge after $\mathbf{str}[3 \dots 3] = a$ and create a new node (u) , followed by a new leaf numbered 3. We then assign the character $\mathbf{str}[4] = c$ as the edge label between the new node (u) and leaf 3.



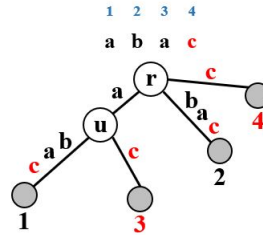
This seems like a rule 2 extension, but in order to add the new leaf we must split an existing edge and insert a new internal node. We define this alternate case of a rule 2 extension as follows.

Definition 9: Rule 2 extensions (case 2)

When performing extension j in phase $i + 1$ first find the path $\text{str}[j \dots i]$ in $\mathcal{I}_i$. If this path **does not** end at a leaf, and the next character in the existing path is some $x \neq \text{str}[i + 1]$, then split the edge after $\text{str}[\dots i]$ and create a new node (u) , followed by a new leaf numbered j and assign the character $\text{str}[i + 1]$ as the edge label between the new node (u) and the leaf j .

**Phase: $i + 1 = 4$, extension $j = 4$**

To insert $\text{str}[4] = c$ we first follow the path $\text{str}[j \dots i] = \text{str}[4 \dots 3]$, which is the empty string from (r) . No path from the root continues with the character $\text{str}[4] = c$, so we make the extension using rule 2 (case 1). We create a new leaf numbered 4 and assign character c as the edge label between the (r) and the leaf 4.



With this final extension we have finished phase $i + 1 = 4$ and have constructed the implicit suffix tree for $\text{str}[1 \dots 4] = abac$.

Question 3: Constructing the regular suffix tree for abac

Append $\text{str}[1 \dots 4] = abac$ with $\$$ and perform one final phase of the above process in order to obtain the regular suffix tree for $\text{str}[1 \dots 4] = abac$.

The naive algorithm described through the previous example can be summarised by the pseudocode below.

```

1  implicit_suffix_tree(str):
2      n = len(str)
3      Construct I_1
4      for i from 1 to n-1:
5          # begin phase i+1
6          for j from 1 to i+1:
7              # begin extension j
8              Follow the path str[j...i] from the root in the current state of the implicit suffix tree.
9              Apply the appropriate suffix extension rule.
10             # str[j...i+1] is now in the tree
11         # end of phase i+1 (I_{i+1} computed)

```

Time complexity of naive (implicit) suffix tree construction

Given a string $\mathbf{str}[1 \dots n]$ the worst case time complexity of the naive suffix tree construction described in this section is $O(n^3)$.

Proof:

Each of the suffix extension rules (rules 1 – 3) can be applied in $O(1)$ -time to extend the path $\mathbf{str}[j \dots i]$ to $\mathbf{str}[j \dots i + 1]$, once the end of the path $\mathbf{str}[j \dots i]$ has been found. In order to find the end of the path $\mathbf{str}[j \dots i]$ naively we must traverse down $O(i + 1 - j)$ characters in the tree. In phase $i + 1$ the extensions $j = 1, 2, \dots, i, i + 1$ are made. Therefore, within phase $i + 1$ we traverse,

$$i + i - 1 + \dots + 2 + 1 = O(i^2),$$

characters in the tree and make $i + 1$ extensions, requiring $O(i^2)$ -time overall for phase $i + 1$. There are $n + 1$ phases in the algorithm numbered $i + 1 = 1, 2, \dots, n, n + 1$ - with the first being the base case and the last being the extension to a regular suffix tree - giving the sequence,

$$1 + 1^2 + 2^2 + \dots + (n - 1)^2 + n^2 = O(n^3),$$

and so overall the algorithm requires $O(n^3)$ -time. ■

Question 4: Proving common sums

Prove the following sums by mathematical induction (or otherwise).

1. $1 + 2 + \dots + (n - 1) + n = \frac{n(n+1)}{2},$
2. $1^2 + 2^2 + \dots + (n - 1)^2 + n^2 = \frac{n(n+1)(2n+1)}{6}.$

Clearly the naive method for constructing a suffix tree by successively growing an implicit suffix tree is practically useless as it requires $O(n^3)$ -time, whereas naively constructing a suffix tree by directly inserting the full suffixes only requires $O(n^2)$ -time. Despite this, the naive method fulfils an important conceptual role as it's high level structure is identical to that of Ukkonen's algorithm. In fact, we will now develop Ukkonen's algorithm by successively improving on this naive method until we obtain the desired linear time bound.

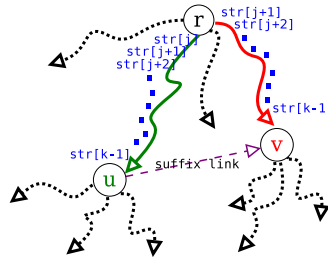
3.3 Speeding up traversal with suffix links

In our analysis of the naive method we noted that in phase $i + 1$ we traverse $O(i^2)$ characters in order to find the necessary extension points. This is because for each extension j we traverse the required path $\text{str}[j \dots i]$ character by character starting from the root. However, the paths that we follow within a phase are related and our first optimisation will provide us with 'shortcuts' within the tree that allow us to efficiently move between the extension points within a phase. In particular, recall that extension j traverses the path $\text{str}[j \dots i]$, while the $j + 1$ extension that immediately follows traverses the path $\text{str}[j + 1 \dots i]$. The characters of these paths are identical except for $\text{str}[j]$ which isn't present in the second path. In light of this revelation we make the following definition.

Definition 10: Suffix links

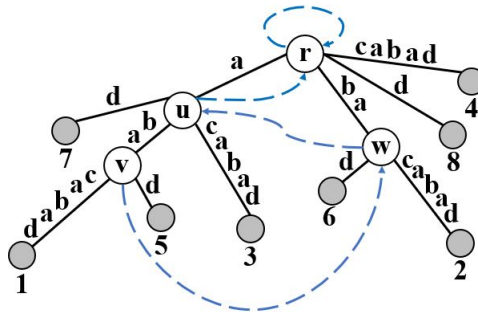
Let (u) and (v) be two internal nodes of an implicit suffix tree. Let the traversal from the root node (r) to (u) yield some substring $\text{str}[j \dots k - 1]$ and the traversal from (r) to (v) yield $\text{str}[j + 1 \dots k - 1]$. We define the *suffix link* between (u) and (v) as a pointer from (u) to (v) . If $(u) = (r)$ then the substring traversed is empty and so $(v) = (r)$ as well, i.e. the root links to itself.

To visualise suffix links it is often helpful break the path $\text{str}[j \dots k - 1]$ up into two sections. Let x be the first character in this path ($\text{str}[j] = x$) and let α denote the remainder of the substring ($\text{str}[j + 1 \dots k - 1] = \alpha$), so that $\text{str}[j \dots k - 1] = x\alpha$. Using this decomposition, if the path to an internal node yields the substring $x\alpha$, then its suffix link points to the internal node whose path yields the substring α .



Example 5: Suffix links

Consider the implicit suffix tree for $\text{str}[1 \dots 8] = \text{abacabad}$ shown below, where the suffix links are shown as blue arrows. The path from the root (r) to (u) is $\text{str}[1] = a$ and the path from the root to itself yields the empty string. In this case $x = a$ and α is the empty string and so we have a suffix link from (u) to (r) . Similarly, the path to (w) yields the substring $\text{str}[6 \dots 7] = ba$ and so we would have $x = b$ and $\alpha = a$. Therefore the suffix link from (w) points to (u) , whose path yields $\text{str}[7] = a$. Finally, the path to (v) yields $\text{str}[5 \dots 7] = aba$ ($x = a$ and $\alpha = ba$) and so its suffix link points to (w) whose path yields $\text{str}[6 \dots 7] = ba$.



Suffix links provide shortcuts that allow us to traverse the tree **without** having to start from the root for each extension. But before we can utilise suffix links we must first traverse to an extension point naively. Fortunately, the first extension $j = 1$ of any phase after phase $i + 1 = 1$ is a rule 1 extension that requires traversing the path labeled by $\text{str}[1 \dots i]$. This fact will turn out to be a consequence of a more general result that we will discuss

shortly, but for now we can rationalise it by noting that $\text{str}[1 \dots i]$ is the longest path in $\mathcal{I}_i$ before any extensions have been performed in phase $i + 1$ and so must end at leaf 1.

In the example shown in figure 5 we would start from the root node (r) , and traverse to the node (v) that is the parent of leaf 1. We would then traverse the remainder of the path below (v) and apply rule 1. We call the node (v) the *active node* and the substring we traverse below it before making the extension ($\text{str}[4 \dots 8] = \text{cabad}$) the *remainder*.

Definition 11: Active node and remainder (substring)

In general, in any extension, the node (v) under whose direct edge the suffix extension rules are applied, is termed the **active node**.

The substring $\text{str}[k \dots i]$ ($\text{str}[4 \dots 8]$ below) that is remaining below the active node (v) and before the extended character $\text{str}[i + 1]$ is termed the **remainder** (substring)

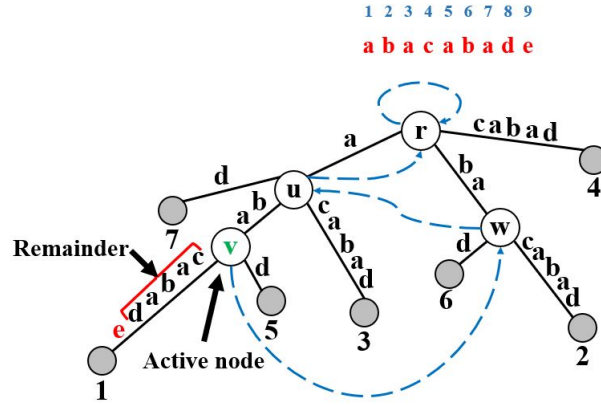


Figure 5: Extension $j = 1$ in phase $i + 1 = 9$ of implicit suffix tree construction for $\text{str}[1 \dots 9] = \text{abacabade}$ requires traversing $\text{str}[1 \dots 8] = \text{abacabad}$, which leads to leaf 1 where a rule 1 extension is performed to add $\text{str}[9] = e$. Here the active node would be v and the remainder is $\text{str}[4 \dots 8] = \text{cabad}$

These two parameters are used to keep track of the *extension point*, which is the point in the tree where the next extension rule should be applied. To see how they are utilised we proceed to extension $j = 2$.

To find the end of $\text{str}[2 \dots i]$ (and extend it by $\text{str}[i + 1]$):

- From the active node (v) , follow its suffix link (shortcut) to (w) . Note that if $(v) = (r)$, then $(w) = (r)$ and we also remove the first character from remainder.
- From (w) , traverse down along the remainder substring $\text{str}[k \dots i]$ ($\text{str}[4 \dots 8] = \text{cabad}$ in figure 6).
- Apply the pertinent suffix extension rule.

General extension procedure for phase $i + 1$

1. Coming into any general extension j from the previous extension $j - 1$, we know the active node $\textcircled{u}$ and the remainder substring $\text{str}[k \dots i]$, which is possibly empty.
2. Follow the suffix link (shortcut) from $\textcircled{u}$ to the receiving node $\textcircled{v}$ and traverse from $\textcircled{v}$ to the end of the remainder substring.
 - Recall that if $\textcircled{u} = \textcircled{r}$ then $\textcircled{v} = \textcircled{r}$, and we must remove the first character from remainder before traversing down.
3. Once the remainder substring has been traversed we are at the end of the path labeled $\text{str}[j \dots i]$, and we apply the pertinent suffix extension rule.

This general procedure enables efficient traversal between extension points by removing the need to traverse the entirety of the path $\text{str}[j \dots i]$ for each extension j . However, we have assumed rather recklessly that the required suffix links will exist in the tree when we need them! Shortly we will prove that this is indeed the case, but we will find it useful to prove the following lemmas.

Lemma 1: Rule 2s are never followed by rule 1s

If extension j of phase $i + 1$ is performed using rule 2 then extension $j + 1$ will not be performed using rule 1.

Proof:

If extension j of phase $i + 1$ was performed using rule 2 then the path $\text{str}[j \dots i]$ does not end at a leaf and can only be continued by characters that are not $\text{str}[i + 1]$. As this path doesn't end at a leaf, it must be a proper prefix of a longer path $\text{str}[k \dots i + 1]$. Note that the longer path finishes at $\text{str}[i + 1]$ as to be longer than $\text{str}[j \dots i]$ it must have been added before extension j ($k < j$) and thus extension k has already been performed in phase $i + 1$. In each phase, the suffixes (of the prefix) are added to the tree in order of descending length, and so immediately after the suffix $\text{str}[k \dots i + 1]$ was added the suffix $\text{str}[k + 1 \dots i + 1]$ must have been added. As $\text{str}[j \dots i]$ is a proper prefix of $\text{str}[k \dots i + 1]$, $\text{str}[j + 1 \dots i]$ must be a proper prefix of $\text{str}[k + 1 \dots i + 1]$, which already exists in the tree. Thus, the path $\text{str}[j + 1 \dots i]$ cannot end at a leaf and therefore extension $j + 1$ will not be performed using rule 1. ■

We have shown that a rule 2 extension is always immediately followed by either a rule 2, or rule 3 in the next extension of the same phase. This is interesting in its own right, but it also hints at something fundamental about the structure of implicit suffix trees.

Lemma 2: There's more structure below the shorter path

The subtree below some path $\text{str}[j \dots i]$ in an implicit suffix tree is a subtree of the subtree below the path $\text{str}[j + 1 \dots i]$. That is, the subtree below $\text{str}[j + 1 \dots i]$ always contains the subtree below $\text{str}[j \dots i]$, but the converse is not true.

Proof:

The proof follows along the same lines as the proof for lemma 1, but we will briefly sketch it here for the sake of completeness.

Assume that the path $\text{str}[j \dots i]$ is added to the tree in extension j of phase i . In the very next extension of the same phase the path $\text{str}[j + 1 \dots i]$ will be added to the tree. Imagine now that in some later phase the path $\text{str}[j \dots i]x$, where x is some non-empty substring was added to the tree. In the very next extension of the same phase the path $\text{str}[j + 1 \dots i]x$ will also be added to the tree. Therefore at the end of any given phase $k > i$ (when we have a completed implicit suffix tree $\mathcal{I}_k$) any path x that exists below $\text{str}[j \dots i]$ must also exist below $\text{str}[j + 1 \dots i]$ and so the subtree below $\text{str}[j \dots i]$ exists below $\text{str}[j + 1 \dots i]$. However, the string may contain multiple copies of the substring $\text{str}[j + 1 \dots i]$ that are not all subsumed by an occurrence

of $\mathbf{str}[j \dots i]$. Furthermore, if such occurrences of $\mathbf{str}[j + 1 \dots i]$ are followed by characters that are different from those that follow the occurrences that are subsumed by an occurrence of $\mathbf{str}[j \dots i]$ then these differing characters will appear in the subtree below $\mathbf{str}[j + 1 \dots i]$, but not below $\mathbf{str}[j \dots i]$. ■

Lemma 2 can be a little difficult to visualise so let's consider the example in figure 8. We can see that the subtree below $\mathbf{str}[1 \dots 3] = aba$ is identical to the subtree below $\mathbf{str}[2 \dots 3] = ba$. This is because every time we extend the subtree below aba we extend the subtree below ba in the same way in the very next extension. In contrast, the subtree below $\mathbf{str}[2 \dots 3] = ba$ exists below $\mathbf{str}[3] = a$, but the latter subtree contains additional structure. This is because every occurrence of $\mathbf{str}[2 \dots 3] = ba$ subsumes $\mathbf{str}[3] = a$, but there are occurrences of $\mathbf{str}[3] = a$ that are not part of an occurrence of $\mathbf{str}[2 \dots 3] = ba$. For instance, both $\mathbf{str}[1]$ and $\mathbf{str}[5]$ are occurrences of a that are not part of an occurrence of ba and this results in an additional branch beginning with b extending from the root of the subtree ($\textcircled{u}$) below $\mathbf{str}[3] = a$. Finally, we note that this additional structure is the reason that a rule 2 in extension j can be followed by a rule 3 in extension $j + 1$, as the absence of the character $\mathbf{str}[i + 1]$ below the path $\mathbf{str}[j \dots i]$ does not imply that $\mathbf{str}[j + 1 \dots i]$ is not continued by $\mathbf{str}[i + 1]$.

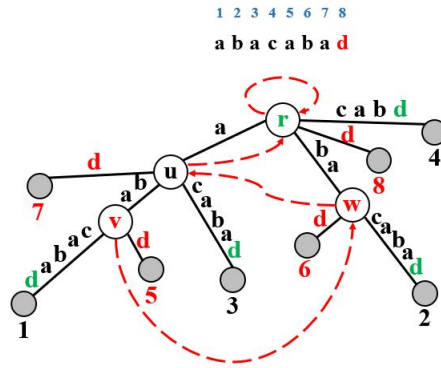


Figure 8: $\mathcal{I}_8$ for the string $\mathbf{str}[1 \dots 8] = abacabad$.

Theorem 1: Existence of suffix links

Every internal node in the implicit suffix tree $\mathcal{I}_i$ has an outgoing suffix link.

Proof:

Let $P(k)$ be the statement that every internal node in the implicit suffix tree $\mathcal{I}_k$ has an outgoing suffix link, we will prove this statement by mathematical induction on k .

Base case:

Let $k = 1$ and consider $\mathcal{I}_1$, which contains a single internal node, the root. By definition the root has an outgoing suffix link that points to itself and so $P(1)$ is true.

Inductive hypothesis:

Assume that $P(i)$ is true, where $i \geq 1$. That is, assume that every internal node in $\mathcal{I}_i$ has an outgoing suffix link.

Inductive step:

In phase $i + 1$ the implicit suffix tree $\mathcal{I}_{i+1}$ is constructed from $\mathcal{I}_i$. By our inductive hypothesis every internal node in $\mathcal{I}_i$ has an outgoing suffix link at the beginning of the phase and no suffix links are ever removed from the tree. Thus, all that remains is to prove that every new internal node created in phase $i + 1$ has an outgoing suffix link by the conclusion of the phase.

A new internal node (v) is only added to the tree via a rule 2 (case 2) extension that splits an existing edge (see example in figure 9). Assume this occurs in extension j . For this to occur the path labeled $\text{str}[j \dots i]$ must have been continued by a character x , such that $x \neq \text{str}[i + 1]$.

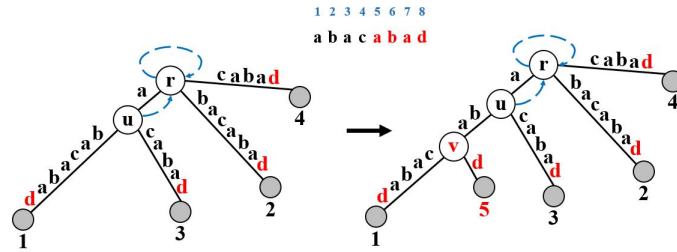


Figure 9: An example of a rule 2 extension adding a new internal node v to the implicit suffix tree.

By lemmas 1 and 2 we know that extension $j + 1$ can only be performed using a rule 2, or a rule 3 and so there are only two cases to consider.

1. The path labeled $\text{str}[j + 1 \dots i]$ continues only via x (i.e. the subtrees are identical) and so extension $j + 1$ will also be performed via rule 2 (case 2) and a new internal node (w) will be created. The path to this new internal node is $\text{str}[j + 1 \dots i]$ and so by definition the outgoing suffix link from (v) points to (w) . See figure 10 for an example.

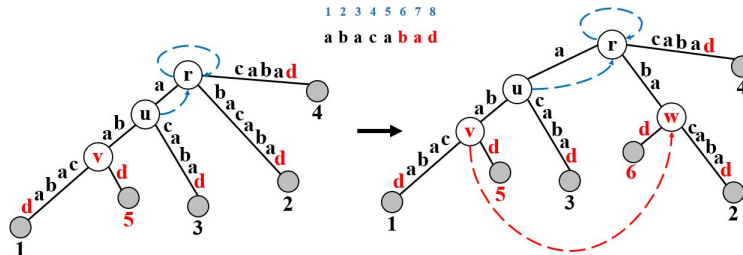


Figure 10: If a new internal node v is created in extension j and a new internal node w is created in extension $j + 1$, then the outgoing suffix link from v points to w .

2. The path labeled $\text{str}[j+1 \dots i]$ ends in an existing internal node $\textcircled{u}$, with one branch extending below via x and one or more branches via other characters, i.e. the subtree under $\text{str}[j+1 \dots i]$ is larger than the subtree under $\text{str}[j \dots i]$. If none of these characters are $\text{str}[i+1]$, then the $j+1$ extension is performed using rule 2 (case 1). Otherwise, if one of the characters is $\text{str}[i+1]$, then the extension is performed using rule 3. In both cases the outgoing suffix link from $\textcircled{v}$ points to $\textcircled{u}$ by definition. See figure 11 for an example.

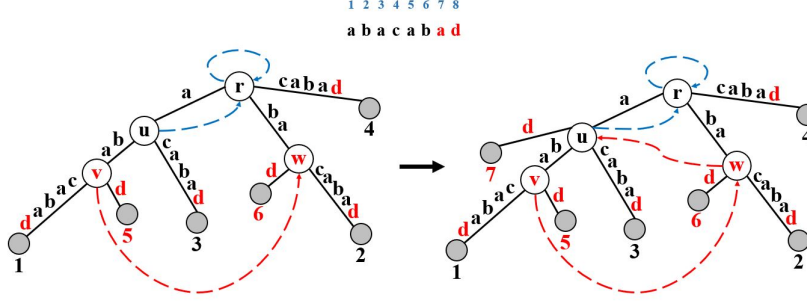


Figure 11: If a new internal node w is created in extension j and the path $\text{str}[j+1 \dots i]$ ends in an existing node u directly below which either a rule 2 (case 1), or rule 3 is performed, then the outgoing suffix link from w points to u by definition.

We have shown that when a new internal node is created its suffix link is resolved in the very next extension of the phase. Finally, we must show that a new node is never created in the last extension of a phase. To do so we simply note that the last extension in any phase $i+1$ traverses the path $\text{str}[i+1 \dots i] = "$ and so the extension is made directly from the root. The suffix $\text{str}[i+1]$ is either added via a rule 2 case 1 extension or via a rule 3 extension and so no new internal node is created.

With this we have shown that by the end of phase $i+1$, every internal node in $\mathcal{I}_{i+1}$ will have an outgoing suffix link and so $P(i+1)$ is true. Therefore, by the principle of mathematical induction $P(k)$ is true for all $k \geq 1$. ■

Theorem 1 proves that the general extension process is sound as the active node is never a newly created internal node and so its outgoing suffix link is always defined. However, even when using suffix links to aid our traversal our method still requires $O(n^3)$ -time as we naively traverse the remainder substring character by character in every iteration.

3.4 Skip-counting

As we know that a path labeled by the remainder substring must exist below the active node it is unnecessary to traverse it character by character. Instead it is more efficient to skip, or jump from node to node along this path only pausing to check the first character of an edge against the current position in the remainder in order to determine which edge continues the path. This process is called skip-counting as we use the number of characters along an edge to determine how far through the remainder traversing it will take us. We skip-count down edges, and possibly through one, or more internal nodes until we reach the extension point.

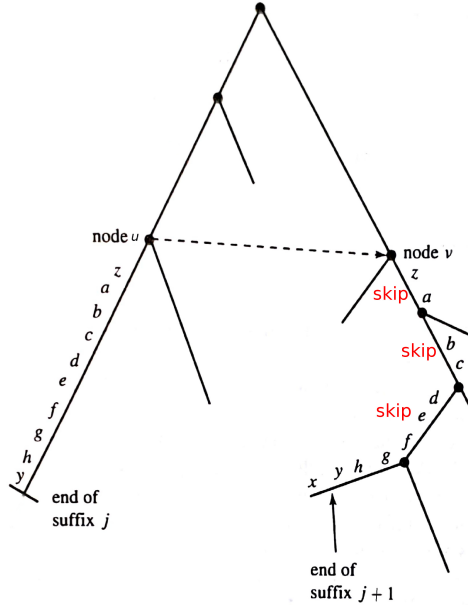
By traversing the remainder using the skip-counting optimisation the time required to traverse this path now depends on the number of nodes it contains rather than the sum of the edge lengths along the path. Using this optimisation in conjunction with suffix links to traverse between extension points reduces the worst case complexity of our method to $O(n^2)$. Although it would appear that so far all we have achieved is the development of an arguably more complex method that offers no benefits over naive suffix tree construction, we have made crucial conceptual headway. And with just 3 more simple optimisations we will further reduce this worst case to the desired $O(n)$ and have fully defined Ukkonen's algorithm.

Example 6: Skip-counting

Imagine that an extension is made below $\textcircled{u}$ and the remainder substring is $\text{str}[k \dots i] = \overset{1}{z} \overset{2}{a} \overset{3}{b} \overset{4}{c} \overset{5}{d} \overset{6}{e} \overset{7}{f} \overset{8}{g} \overset{9}{h}$
 $\overset{10}{y}$, as shown below. We then traverse the suffix link out of $\textcircled{u}$ to $\textcircled{v}$ and traverse down the remainder using skip-counting as follows:

- From node $\textcircled{v}$ determine how many characters are on the edge beginning with $z \dots$. Here, it is 2.
- The length of the remainder substring is 10 and since $10 > 2$, traversing this edge will not reach the end of remainder so skip to the node receiving that edge.
- In the remainder the 3rd ($3 = 2 + 1$) character is b .
- Again, ask how many characters lie on the edge starting with b , there are 2.
- By traversing this edge we will have traversed $2 + 2 = 4$ characters of the remainder, and since $10 > 4$ we must continue to traverse down the tree.
- The 5th ($5 = 2 + 2 + 1$) character in the remainder is d .
- Again, ask how many characters lie on the edge starting with d , there are 3.
- Since $10 > 2 + 2 + 3$, it is safe to skip to the node receiving this edge.
- Finally, the 8th ($8 = 2 + 2 + 3 + 1$) character in the remainder is g , and so from the current node we find the edge beginning with g . This edge is labeled by 4 characters and so traversing the entire edge would take us beyond the end of the remainder substring. Thus, we simply traverse partway down this edge by skipping down 3 characters ($10 - 2 - 2 - 3 = 3$), to reach the extension point. The appropriate extension can then be performed.

Note: while skip-counting down the remainder substring the active node should be continually updated to the most recent (or deepest) internal node visited. This ensures that after the traversal is complete the active node will be the internal node immediately above the extension point as required.



3.5 Space efficient representation of edge labels

The first barrier we must overcome in our quest to better our current $O(n^2)$ method is that the representation of our implicit suffix trees requires $O(n^2)$ -space in the worst case. Such a representation prohibits the development of a linear time method as we would be unable to even output the resulting tree in $O(n)$ -time. Our current representation requires $O(n^2)$ -space because we explicitly store the edge labels in the tree. However, we have already seen in the introduction to this section that this is not necessary and we can instead store each of these substrings using just two integers, *startIndex* and *endIndex* as shown in figure 12. We can think of *startIndex* and *endIndex* as pointers into the original string **str**[1...*n*] that define the start and end position of the substring, respectively. Thus, the space required to store each edge in the tree is reduced to $O(1)$ and overall this representation requires $O(n)$ -space. We will assume that all implicit suffix trees from this point forward are represented in this manner, but will continue to include the explicit edge labels for the sake of clarity.

Theorem 2: Space efficient suffix trees

Given a string **str**[1...*n*] defined over a fixed size alphabet $\aleph$, the worst case space complexity of representing a suffix tree, or an implicit suffix tree using the efficient edge labels is $O(n)$.

Proof:

If you've completed question 2 this proof follows trivially, but we will reproduce it here.

Let's assume that every node uses an array of size $\aleph$ to store its edge list. Therefore, the array contains one element for every possible edge that could be added to the node and we require $O(\aleph)$ -space to store each node. In addition, we know that when using our efficient edge representation method we require $O(1)$ -space to store each edge. Due to this, to upper bound the space required to store the tree in the worst case we simply need to bound the number of edges and nodes that could be present in the tree.

We know that number of nodes and edges in any implicit suffix tree for **str** is upper bounded by the number of nodes and edges in the regular suffix tree for **str**. Moreover, we know that the regular suffix tree contains *n* leaves and every internal node must have at least two edges. This means that the size of our tree is bounded by the size of a perfect binary tree that also has *n* leaves. The number of internal nodes at each level in such a binary tree forms a geometric series with sum,

$$\frac{n}{2} + \frac{n}{4} + \dots + 2 + 1 = n - 1,$$

giving $2n - 1$ nodes overall.

Every node - aside from the root - has exactly one incoming edge and so the number of edges in a perfect binary tree is $2n - 2$. Thus, the space required to store the nodes in the suffix tree for **str** is upper bounded by $(2n - 1) \times O(\aleph) = O(n)$ for a fixed size alphabet and the space required to store the edges is upper bounded by $(2n - 2) \times O(1) = O(n)$, giving $O(n)$ -space in the worst case. ■

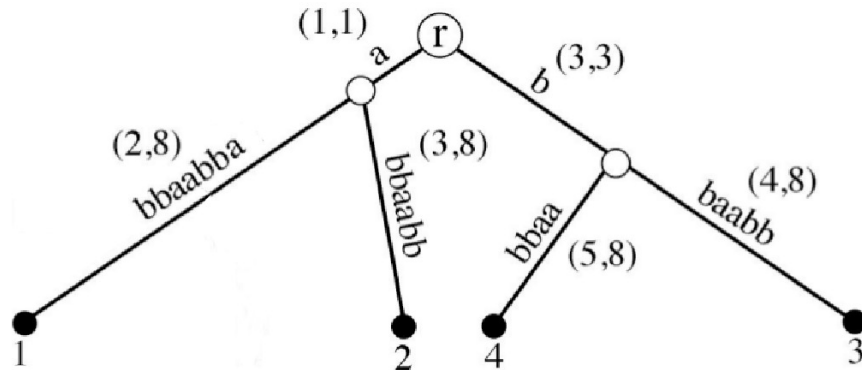


Figure 12: An implicit suffix tree of the string **str**[1...8] = aabbaabb, represented using the (*startIndex*, *endIndex*) edge label representation.

3.6 Extension termination criterion (the showstopper rule)

Now that we are able to represent our implicit suffix trees in $O(n)$ -space it is possible to continue progressing towards a linear time construction method. The remaining optimisations will focus on reducing the number of extensions we make in a phase, as this now fundamentally limits the time required by the algorithm. Specifically, as each phase $1 \leq i + 1 \leq n$ requires $1 \leq j \leq i + 1$ extensions, overall we must perform $O(n^2)$ extensions in the algorithm. Obviously, we cannot explicitly perform all these extensions if we are to obtain an $O(n)$ -time method. Fortunately, both optimisations are direct consequences of simple observations.

Lemma 3: Rule 3s are followed by rule 3s

If in any phase $i + 1$ the extension j is performed via rule 3, then every extension $(j + 1, j + 2, \dots, i + 1)$ for the remainder of the phase will be performed using rule 3.

Proof:

This is a simple consequence of lemma 2. We know that if a rule 3 occurs in extension j then the path $\text{str}[j \dots i]$ must be continued by $\text{str}[i + 1]$. However, we have already know that any character x that continues the path $\text{str}[j \dots i]$, must also continue the path $\text{str}[j + 1 \dots i]$. Thus, if extension j was performed using rule 3, then extension $j + 1$ will also be performed using rule 3. Similarly, if extension $j + 1$ was performed using rule 3, then extension $j + 2$ will also be performed using rule 3 and so on until the end of the phase at $j = i + 1$. ■

To leverage lemma 3, we note that rule 3 extensions require us to make no alterations to the tree and so we can define the *showstopper rule*.

Definition 12: Showstopper rule

If any extension j in phase $i + 1$ is performed using rule 3, then immediately terminate the phase and begin computing extensions for phase $i + 2$.

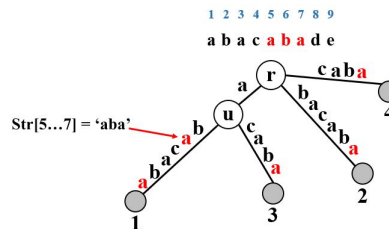
The validity of the showstopper rule is easily rationalised. Lemma 3 tells us that if extension j in a phase $i + 1$ is a rule 3 extension, then all remaining extensions in the same phase will also be rule 3 extensions. As each of these rule 3 extensions does not alter the state of the tree, it will remain unchanged until the first extension of the next phase (phase $i + 2$), which will always be a rule 1 after the base case. Therefore we can avoid traversing to all the extension points for the extensions $j + 1, j + 2, \dots, i + 1$ in phase $i + 1$ and simply move to the first extension of phase $i + 2$.

Example 7: The showstopper rule

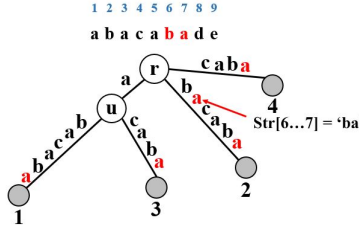
To see the showstopper rule in action let's consider phase $i + 1 = 7$ of the implicit suffix tree construction for,

$\text{str}[1 \dots 9] = \overset{1}{a} \overset{2}{b} \overset{3}{a} \overset{4}{c} \overset{5}{a} \overset{6}{b} \overset{7}{a} \overset{8}{d} \overset{9}{e}$.

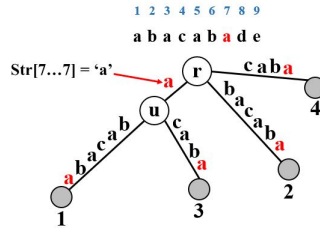
Extension $j = 5$ in this phase is the first occurrence of a rule 3 extension and we are attempting to insert $\text{str}[5 \dots 7] = aba$, which already exists as shown below. Intuitively this path already exists in the tree as this suffix (of the prefix $\text{str}[1 \dots 7]$) is a proper prefix of the longer suffix $\text{str}[1 \dots 7]$. This longer suffix was added in an earlier extension ($j = 1$).



Similarly, in extension $j = 6$ we are attempting to insert $\text{str}[6 \dots 7] = ba$, which already exists in the tree as it is a proper prefix of the longer suffix $\text{str}[2 \dots 7]$ as shown below.



Finally, in extension $j = 7$ we are attempting to insert $\text{str}[7] = a$, which already exists in the tree as it is a proper prefix of the longer suffix $\text{str}[3 \dots 7]$ as shown below.



We can see that after the first rule 3 extension of the phase all the remaining extensions were also performed using rule 3. Due to this, we could've immediately moved onto extension $j = 1$ of phase $i + 1 = 8$ after extension $j = 5$.

3.7 Rapid leaf extension

The showstopper rule is enabled by the fact that when a rule 3 extension occurs we know that all the extensions for the remainder of the phase will also be performed using rule 3. The following two lemmas will let us make an analogous statement for the rule 1 extensions.

Lemma 4: Rule 1s are rule 1s in the next phase

If extension j of phase i is performed using rule 1, then the same extension j in the next phase $i + 1$ will also be performed using rule 1.

Proof:

If extension j of phase i is a rule 1 extension the path $\text{str}[j \dots i - 1]$ must end at leaf j . This path is then extended to $\text{str}[j \dots i]$ by extension j . Recall that while rule 2 extensions create new leaves there are no operations that remove or alter the path to existing leaves, aside from rule 1 extensions which simply extend this path by one character. Every extension after extension j in phase i traverses a shorter path than $\text{str}[j \dots i]$ and so will never reach leaf j . Similarly, every extension **before** extension j in phase $i + 1$ will traverse a longer path than $\text{str}[j \dots i]$ and so will never reach the leaf j . Thus, by extension j in phase $i + 1$ the path $\text{str}[j \dots i]$ still exists in the tree and still ends at leaf j . As our path once again ends at a leaf, extension j of phase $i + 1$ will be performed using rule 1. ■

Lemma 5: Rule 2s are rule 1s in the next phase

If extension j of phase i is performed using rule 2, then the same extension j in the next phase $i + 1$ will be performed using rule 1.

Proof:

If extension j in phase i is performed using rule 2 then this extension creates a leaf numbered j whose path from the root is $\mathbf{str}[j \dots i]$. By the same logic presented in the proof of lemma 4, the path $\mathbf{str}[j \dots i]$ is unaltered until extension j of the next phase $i + 1$. This extension then traverses the full path $\mathbf{str}[j \dots i]$ to leaf j and extends it to $\mathbf{str}[j \dots i + 1]$ using rule 1. ■

The regular suffix tree for $\mathbf{str}[1 \dots n]$ contains n leaves, one for each suffix. The only extension that creates a leaf is a rule 2 and the only extension that updates the path to a leaf is rule 1. Intuitively lemma 5 tells us that the suffix $\mathbf{str}[j \dots n]$ is added to the implicit suffix tree using a rule 2 in extension j of some phase i . In combination with lemma 4, we can see that the prefix $\mathbf{str}[j \dots i]$ of the full suffix $\mathbf{str}[j \dots n]$ is then grown by one new character in each successive phase via rule 1s. This continues until phase n where the tree implicitly contains the full suffix $\mathbf{str}[j \dots n]$. That is, rule 2s add new suffixes to the tree, and rule 1s grow the suffixes that already exist in our tree. Furthermore, once a suffix is added to the tree it is never added again and so if extension j in phase i is a rule 2, we will never perform a rule 2 for extension j in any phase $k > i$. In fact, by lemmas 4 and 5 we know that extension j for any phase $k > i$ will be performed using rule 1.

To help cement this realisation it is worth revisiting a previous example. For instance, you may want to return to the example in section 3.2 and observe extension 1 of each phase. In phase 1 (the base case) this is a rule 2 extension that adds the suffix $\mathbf{str}[1]$ of the prefix $\mathbf{str}[1]$. For every phase $i + 1 > 1$ extension 1 is performed using rule 1, and the suffix is grown into the full suffix $\mathbf{str}[1 \dots 4]$ by the conclusion of phase $i + 1 = 4$. Now that we know which extensions in a phase will be rule 1s we can develop a method to exploit this.

First we need to keep track of which extensions in the current phase i will be rule 1 extensions. In other words, we need to know how many extensions were performed using rule 2 before phase i . Let's define a variable $last_j$ to keep track of this.

Definition 13: $last_j$

Let $last_j$ be the last extension j that was performed using rule 2.

Every extension $1 \leq j \leq last_j$ of phase i will be performed using rule 1. However, we need to develop some way to perform these extensions without explicitly traversing the path to each leaf in the current tree.

Once a leaf j is added to the tree it is never altered. The edge to the leaf may be split by rule 2 extensions and the character $\mathbf{str}[i]$ will be added to the end of the edge in phase i . Recalling that the edge to the leaf will be stored as $(startIndex, endIndex)$, in each phase we simply increment the $endIndex$ for this edge. But $endIndex = i$ for every leaf j in $\mathcal{I}_i$, i.e. the path to the leaf j must be $\mathbf{str}[j \dots i]$. As each edge label has the same $endIndex$ let's store a pointer to a variable $globalEnd$, such that the label for each edge to a leaf is of the form, $(startIndex, globalEnd)$. Now we can perform every rule 1 extension in phase i by simply setting $globalEnd = i$. This will implicitly update all the required edge labels without us ever needing to traverse to any of the leaves! That is, we can perform the first $last_j$ extensions in phase i in $O(1)$ -time, by simply updating $globalEnd$. We then start *explicit extensions* (extensions where we must traverse to the extension point) in phase i from $j = last_j + 1$. Let's summarise what we've found.

Rapid leaf extension

Given an input string $\mathbf{str}[1 \dots n]$, in Ukkonen's algorithm all rule 1 extensions are performed implicitly in the following manner.

- Create variables $last_j = 0$ and $globalEnd = 0$.
- At the beginning of each phase $1 \leq i + 1 \leq n$ increment $globalEnd$, i.e. $globalEnd = globalEnd + 1$.
- Begin explicit extensions from $j = last_j + 1$.

- If an explicit extension j is performed using rule 2:
 - Create a leaf numbered j and attach it to the appropriate internal node. Set $endIndex = globalEnd$ for the edge to the leaf.
 - Increment $last_j$, i.e. $last_j = last_j + 1$.

The $endIndex$ should actually be set to a pointer to $globalEnd$.

3.8 Rule 3s require a little more than nothing

Before concluding we should note that there exists a result for rule 3 extensions that is analogous to lemmas 4 and 5.

Lemma 6: Rule 3s are rule 2s, or rule 3s in the next phase

If extension j of phase i is performed using rule 3, then the same extension j in the next phase $i + 1$ will be performed using either rule 2, or rule 3.

Question 5: Proving lemma 6

Prove lemma 6.

Although we cannot exploit lemma 6 to develop an optimisation, it does provide the final piece of intuition for the role of the extension rules in Ukkonen's algorithm. Intuitively it says that a rule 3 extension is essentially work waiting to happen. Firstly, we **do not** increment $last_j$ after a rule 3 extension, and by the showstopper rule we immediately begin the next phase. So if extension j of phase i is a rule 3 extension the next explicit extension we make will be extension j in phase $i + 1$. However, extension j in phase i traverses the path $\mathbf{str}[j \dots i - 1]$, while the same extension in phase $i + 1$ traverses the path $\mathbf{str}[j \dots i]$. Recall that we only traverse the remainder substring $\mathbf{str}[k \dots i - 1]$ below the active node $\textcircled{u}$ by skip-counting in extension j of phase i . Furthermore, we **do not** traverse a suffix link after a rule 3 extension. In light of this it would appear that simply moving to extension j of phase $i + 1$ will leave us one character short of the desired extension point $\mathbf{str}[j \dots i]$, as the path to $\textcircled{u}$ ($\mathbf{str}[j \dots k - 1]$) is continued by the remainder $\mathbf{str}[k \dots i - 1]$. Thus, even though a rule 3 tells us to do nothing, we must perform a few minor operations before starting the next phase.

Rule 3 extensions in Ukkonen's

If extension j of phase i is performed using a rule 3 extension we perform the following steps:

1. Add the character $\mathbf{str}[i]$ to the remainder to extend it from $\mathbf{str}[k \dots i - 1]$ to $\mathbf{str}[k \dots i]$.
2. If the previous extension j created a new internal node $\textcircled{v}$, point the outgoing suffix link from $\textcircled{v}$ to the current active node $\textcircled{u}$.
3. Terminate phase i using the showstopper rule.

3.9 Summary

After a long journey we have finally reached the point where we are ready to put all the pieces together to form Ukkonen's algorithm. The pseudocode below details at a high level how the algorithm proceeds and in section 3.10 we go through the construction of a suffix tree using Ukkonen's algorithm.

```
1  ukkonen(str):
2      # Append the terminal character
3      str = str + $
4      n = len(str)
5      # Base case
6      construct I_1
7      # Initialise variables, active node, remainder, last_j, globalEnd
8      AN = root
9      rem = None
10     lastJ = 1
11     globalEnd = 1
12     for i from 1 to n-1:
13         # Begin phase i+1 and perform rapid leaf extension
14         globalEnd = globalEnd + 1
15         # Explicit extensions start from lastJ+1
16         for j from lastJ+1 to i+1:
17             # Skip-count down to the extension point
18             traverse()
19             # Apply the appropriate extension
20             makeExtension()
21             # Resolve any pending suffix links from the previous extension
22             resolveSuffixLinks()
23             # Move to next extension i.e. traversing suffix links, or showstopper
24             moveToNextExtension()
25         # str[j...i+1] is now in the tree
26     # End of phase i+1 (I_{i+1} computed)
27     # Return the tree
28     return root
```

Implementation notes

The exact implementation of the algorithm will be left to the reader, but it is worth making some brief comments on how one should proceed.

- Python does not support pointers, but the same behaviour can be achieved using mutable data types such as lists, implementing a class, or through other methods.
- The remainder should not be stored as an explicit substring, instead it should be stored using references into the original string.
- When traversing to the extension point by skip-counting down the remainder, the active node and remainder will need to be updated if the traversal moves through one or more internal nodes.
- The variables active node, remainder, $last_j$ and $globalEnd$ should be updated appropriately following an extension.
- If a new internal node is created its suffix link should be resolved in the next extension.
- After a rule 2 we must traverse the outgoing suffix link from the active node. The receiving node becomes the new active node.
- The showstopper rule means that not all the extensions $last_j + 1 \leq j \leq i + 1$ are necessarily performed in phase $i + 1$.

Worst case space and time complexity of Ukkonen's algorithm

Given a string $\mathbf{str}[1 \dots n]$, Ukkonen's algorithm requires $O(n)$ time and space to construct the regular suffix tree for $\mathbf{str}$.

Proof: By theorem 2 the space required to represent any intermediate state of the (implicit) suffix tree in Ukkonen's is bounded by $O(n)$. In addition, the algorithm only requires the storage of a constant number of auxiliary variables and so requires $O(n)$ -space overall. Similarly, to prove that the algorithm requires $O(n)$ -time we simply sum the time required by every operation across the entire algorithm.

Firstly, $n + 1$ phases are required to construct the regular suffix tree for $\mathbf{str}$, where the final phase ($i + 1 = n + 1$) adds the terminal character $\$$ to every suffix in the implicit suffix tree $\mathcal{I}_n$. All the rule 1 extensions in any given phase are performed implicitly by incrementing $globalEnd$, which requires $O(1) \times (n + 1) = O(n)$ time over $n + 1$ phases. As a consequence of lemmas 4 and 5, a rule 2 extension occurs exactly once for each suffix in $\mathbf{str}[1 \dots n]\$$. This means that there are $n + 1$ rule 2 extensions overall, each of which can be performed in $O(1)$ -time from the extension point, contributing $O(n)$ -time overall. Similarly, due to the showstopper rule, there can be at most one rule 3 extension in every phase, contributing $O(n)$ -time overall. Therefore, every extension across the entire algorithm can be performed in $O(n)$ -time. All that remains is to prove that the time spent traversing between explicit extension points is $O(n)$.

As rule 3 extensions do not increment $last_j$ two consecutive phases can share at most 1 explicit extension. That is, if phase i is terminated by the showstopper rule in extension j , the explicit extensions in phase $i + 1$ resume from j and by lemma 6 this extension will either be performed using rule 2, or rule 3. If it is a rule 2 extension we increment $last_j$ ($last_j = j$) and this extension will be handled via rapid leaf extension for every phase $k > i + 1$. If instead extension j in phase $i + 1$ is again performed using rule 3, then we terminate the phase and the process repeats. Thus, overall we perform at most $2(n + 1)$ explicit extensions ($n + 1$ rule 2s and $n + 1$ rule 3s). Once extension j has been completed in some phase using rule 2, we move to the next extension point by traversing the outgoing suffix link from the current active node (u) and then traverse the remainder below the receiving node (v) . This means that the number of suffix links we traverse is bounded by $n + 1$, and each traversal can be performed in $O(1)$ -time, giving $O(n)$ -time overall. Finally let's quantify how much time is spent skip-counting down the remainder.

Imagine that we have completed some extension j and we are moving to the next extension $j + 1$. To do so we follow the suffix link out of the active node (u) - whose path from the root yields $\mathbf{str}[j \dots P_k]$ - to some other internal node (v) . By definition of the suffix link from (u) , the path from the root to (v) must yield $\mathbf{str}[j + 1 \dots P_k]$, as shown below. We then traverse the remainder by skip-counting, and in doing so we may traverse zero, or more internal nodes below (v) in pursuit of the $j + 1$ extension. Say that extension is performed directly below some internal node (w) whose path from the root yields,

$$\mathbf{str}[j + 1 \dots p_k \dots p_{k+1}],$$

where $p_k + 1 \geq p_k$, with equality occurring if we traverse zero internal nodes below (v) , as in this case (v) and (w) are the same node. After extension $j + 1$ is complete we prepare for extension $j + 2$ by following the suffix out of (w) to some other internal node (x) , whose path from the root yields,

$$\mathbf{str}[j + 2 \dots p_k \dots p_{k+1}].$$

We once again traverse the remainder below (x) by skipping down zero or more internal nodes until we reach the internal node (y) on whose outgoing edge extension $j + 2$ is performed. The path from the root to (y) would yield,

$$\mathbf{str}[j + 2 \dots p_k \dots p_{k+1} \dots p_{k+2}],$$

where $p_k \leq p_{k+1} \leq p_{k+2}$. Note that we have made no reference to phases here as extension $j + 1$ may not be performed in the same phase as extension j (they might not even be performed in successive phases). However, we should note that if extension j in one phase is performed using rule 3 we **do not** traverse a suffix link and would simply traverse the remainder below the current active node. In this case we would just traverse the new remainder (it is updated after a rule 3 extension) possibly traversing through new internal nodes until

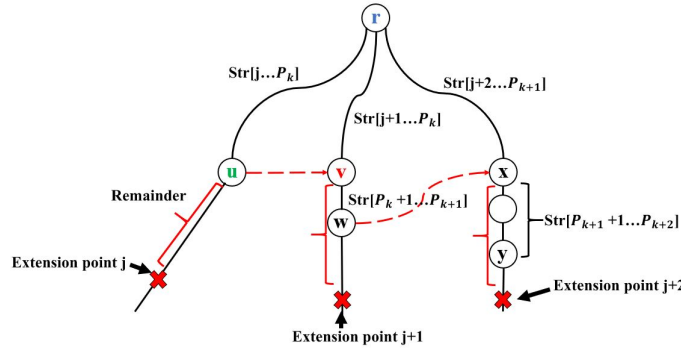
we reach a phase in which extension j is performed using rule 2. We would then follow the suffix link out of the current active node - which is not necessarily the original active node - and pursue the $j + 1$ extension.

The previous process will yield a sequence of indices in **str** of the form,

$$1 \leq p_0 \leq p_1 \leq \dots \leq p_k \leq p_{k+1} \leq p_{k+2} \leq \dots n,$$

where there are at most $2(n + 1)$ terms in the sequence, one for each explicit extension. As our traversal is performed by skip-counting we need to determine how many nodes we traverse through in total. In the worst case, each internal node in the tree is separated by a single character. In this case the distance between two elements of the sequence gives the number of internal nodes traversed between the corresponding explicit extensions. If we let p_m be the last element in the sequence, then $p_m - p_0$ upper bounds the number of internal nodes we traverse through overall. This difference is bounded by n and as skipping through a single internal node requires $O(1)$ -time, our traversal requires $O(n)$ -time overall in the worst case.

In summary, performing all the extensions requires $O(n)$ -time in total and traversing to all the required extension points also requires $O(n)$ -time giving an $O(n)$ -time algorithm overall. ■

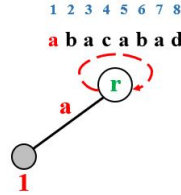


3.10 Extended example

Ukkonen's algorithm can seem rather complex due to the numerous components that all work in concert to achieve the desired linear time bound. Despite this, the actual mechanics of the algorithm are simple to follow and best elucidated by example and so we will conclude our examination of Ukkonen's algorithm by discussing how it will construct the suffix tree for $\text{str}[1 \dots 8] = \text{abacabad}$. The values of the phase, extension, active node (AN), remainder substring (rem), $last_j$ and global pointer GP are displayed **before** each extension and then the updates that are made are noted in the discussion of the extension. All edges will be labeled explicitly with the corresponding substring for the sake of clarity, but an actual implementation would make use of the compressed edge labels discussed earlier. Finally, all skip-counting at the start of each explicit extension starts from the active node **not** from the root (unless it is the active node).

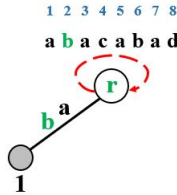
Phase: $i + 1 = 1$, **extension:** $j = 1$, $AN = \text{None}$, $rem = \text{None}$, $last_j = \text{None}$, $GP = \text{None}$

This is the base case of the algorithm and can be handled separately from all other phases. In particular the values of the active node, remainder, $last_j$, and global pointer haven't been defined and so we simply create a new root node ($\textcircled{r}$) and leaf node numbered 1. The edge between the root and leaf 1 is labeled with $\text{str}[1] = a$. Recalling that the base case can be thought of as a rule 2 extension we must increment $last_j$ ($last_j = 1$) and the active node becomes the root ($AN = \textcircled{r}$), while the remainder substring remains empty ($rem = ''$). Finally, the global pointer is set to 1 ($GP = 1$).



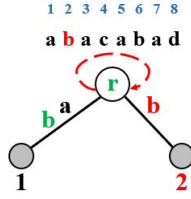
Phase: $i + 1 = 2$, **extension:** $j = 1$, $AN = \textcircled{r}$, $rem = ''$, $last_j = 1$, $GP = 1$

As this is the start of phase 2 we first increment the global pointer so that $GP = 2$. This implicitly performs extension $j = 1$ which extends $\text{str}[1] = a$ to $\text{str}[1 \dots 2] = ab$.



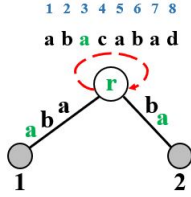
Phase: $i + 1 = 2$, **extension:** $j = 2$, $AN = \textcircled{r}$, $rem = ''$, $last_j = 1$, $GP = 2$

This is the beginning of explicit extensions as $j = last_j + 1 = 2$ and we first skipcount down to the extension point indicated by the active node and remainder. As $AN = \textcircled{r}$ and $rem = ''$ the extension is performed directly from the root. The new character $\text{str}[2] = b$ is added via rule 2 (case 1) which creates a new leaf numbered 2 and is labeled with b . As this is a rule 2 extension we increment $last_j$ to $last_j = 2$ and we follow the suffix link from the active node, which leaves us at the root and we reduce the remainder substring by 1 character. Thus, the active node and the remainder both remain unchanged.



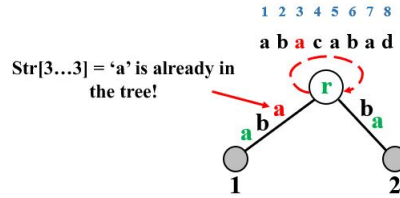
Phase: $i + 1 = 3$, **extension:** $j = 1 - 2$, $AN = (r)$, $rem = ''$, $last_j = 2$, $GP = 2$

As this is the start of phase 3 we first increment the global pointer so that $GP = 3$. This implicitly performs extensions $j = 1 - 2$ which extend, $\mathbf{str}[1 \dots 2] = ab$ to $\mathbf{str}[1 \dots 3] = aba$ and $\mathbf{str}[2] = b$ to $\mathbf{str}[2 \dots 3] = ba$ respectively.



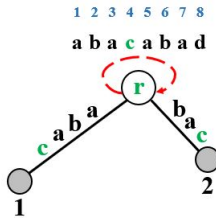
Phase: $i + 1 = 3$, **extension:** $j = 3$, $AN = (r)$, $rem = ''$, $last_j = 2$, $GP = 3$

This is the beginning of explicit extensions as $j = last_j + 1 = 3$ and we first skipcount down to the extension point indicated by the active node and remainder. As $AN = (r)$ and $rem = ''$ the extension is performed directly from the root. The new character $\mathbf{str}[3] = a$ is already in the tree and so the extension is performed via rule 3. As this was not a rule 2 extension $last_j$ remains unchanged and we do not follow any suffix links. The only operation we must perform before ending the phase is to update the remainder to $rem = a$ as we have now traversed part way down the tree.



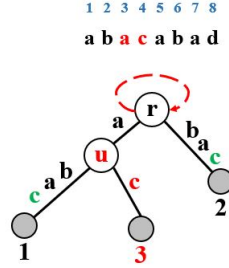
Phase: $i + 1 = 4$, **extension:** $j = 1 - 2$, $AN = (r)$, $rem = a$, $last_j = 2$, $GP = 3$

As this is the start of phase 4 we first increment the global pointer so that $GP = 4$. This implicitly performs extensions $j = 1 - 2$ which extend, $\mathbf{str}[1 \dots 3] = aba$ to $\mathbf{str}[1 \dots 4] = abac$ and $\mathbf{str}[2 \dots 3] = ba$ to $\mathbf{str}[2 \dots 4] = bac$ respectively.



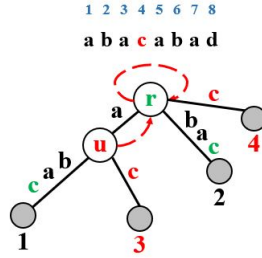
Phase: $i + 1 = 4$, **extension:** $j = 3$, $AN = (\mathbf{r})$, $rem = a$, $last_j = 2$, $GP = 4$

This is the beginning of explicit extensions as $j = last_j + 1 = 3$ and we first skipcount down to the extension point indicated by the active node and remainder. As $AN = (\mathbf{r})$ and $rem = a$ the extension is performed after the a on the edge between $(\mathbf{r})$ and leaf 1. The character immediately after the active point is a b and so the new character $\mathbf{str}[4] = c$ must be added via a rule 2 that splits the edge and creates a new internal node $(\mathbf{u})$ and a new leaf numbered 3. An edge is created between $(\mathbf{u})$ and leaf 3 and is labeled with $\mathbf{str}[4] = c$. We now increment $last_j$ to $last_j = 3$ and follow the suffix link out of the active node which leaves us at the root still. However, as we followed the suffix link out of the root we must manually reduce the remainder substring by giving $rem = ''$.



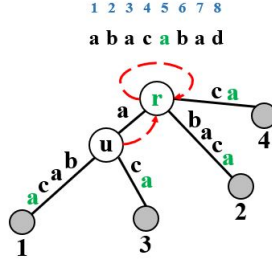
Phase: $i + 1 = 4$, **extension:** $j = 4$, $AN = (\mathbf{r})$, $rem = ''$, $last_j = 3$, $GP = 4$

We first skipcount down to the extension point indicated by the active node and remainder. As $AN = (\mathbf{r})$ and $rem = ''$ the extension is performed directly from the root. No path from the root begins with a c and so the new character $\mathbf{str}[4] = c$ is added via rule 2 (case 1) which creates a new leaf numbered 4 and is labeled with c . On the previous extension we created a new internal node $(\mathbf{u})$ whose suffix link needs to be resolved. In this case we did not create another internal node as the extension was made directly from the active node and so we simply point the suffix link from $(\mathbf{u})$ to the active node which is the root. Finally, as this is a rule 2 extension we increment $last_j$ to $last_j = 4$ and we follow the suffix link from the active node, which leaves us at the root and we reduce the remainder substring by 1 character. Thus, the active node and the remainder both remain unchanged.



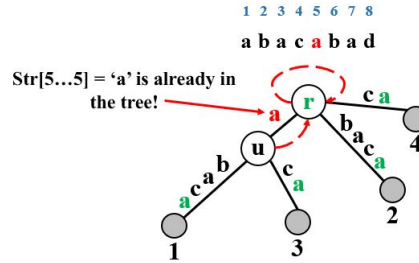
Phase: $i + 1 = 5$, **extension:** $j = 1 - 4$, $AN = (\mathbf{r})$, $rem = ''$, $last_j = 4$, $GP = 4$

As this is the start of phase 5 we first increment the global pointer so that $GP = 5$. This implicitly performs extensions $j = 1 - 4$ as shown below.



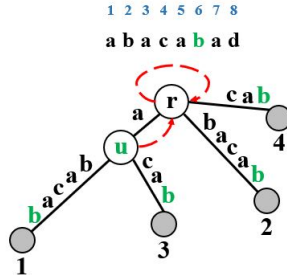
Phase: $i + 1 = 5$, **extension:** $j = 5$, $AN = (\mathbf{r})$, $rem = ''$, $last_j = 4$, $GP = 5$

This is the beginning of explicit extensions as $j = last_j + 1 = 5$ and we first skipcount down to the extension point indicated by the active node and remainder. As $AN = (\mathbf{r})$ and $rem = ''$ the extension is performed directly from the root. There is an edge from the root beginning with an a and so the new character $str[5] = a$ is added via a rule 3 extension. We update the remainder to $rem = a$, and then terminate the phase.



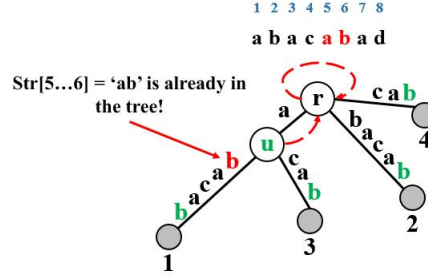
Phase: $i + 1 = 6$, **extension:** $j = 1 - 4$, $AN = (\mathbf{r})$, $rem = a$, $last_j = 4$, $GP = 5$

As this is the start of phase 6 we first increment the global pointer so that $GP = 6$. This implicitly performs extensions $j = 1 - 4$ as shown below.



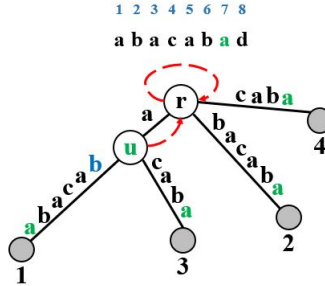
Phase: $i + 1 = 6$, **extension:** $j = 5$, $AN = (\mathbf{r})$, $rem = a$, $last_j = 4$, $GP = 6$

This is the beginning of explicit extensions as $j = last_j + 1 = 5$ and we first skipcount down to the extension point indicated by the active node and remainder. As $AN = (\mathbf{r})$ and $rem = a$ we skip down to $(\mathbf{u})$ at which point we update the active node so that $AN = (\mathbf{u})$ and the remainder is set to $rem = ''$ as there are no more characters to traverse below $(\mathbf{u})$ to reach the extension point. One of the edges that start at $(\mathbf{u})$ begin with a b and so the new character $str[6] = b$ is added via a rule 3 extension. We update the remainder to $rem = b$, and then terminate the phase.



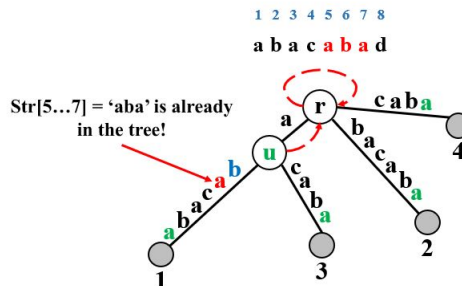
Phase: $i + 1 = 7$, **extension:** $j = 1 - 4$, $AN = (\mathbf{u})$, $rem = b$, $last_j = 4$, $GP = 6$

As this is the start of phase 7 we first increment the global pointer so that $GP = 7$. This implicitly performs extensions $j = 1 - 4$ as shown below.



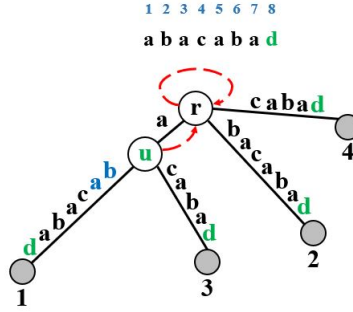
Phase: $i + 1 = 7$, **extension:** $j = 5$, $AN = (\mathbf{u})$, $rem = b$, $last_j = 4$, $GP = 7$

This is the beginning of explicit extensions as $j = last_j + 1 = 5$ and we first skipcount down to the extension point indicated by the active node and remainder. As $AN = (\mathbf{u})$ and $rem = b$ we make the extension directly below the b on the edge between $(\mathbf{u})$ and leaf 1. The character immediately following the extension point is a and so the new character $str[7] = a$ is added via a rule 3 extension. We update the remainder to $rem = ba$, and then terminate the phase.



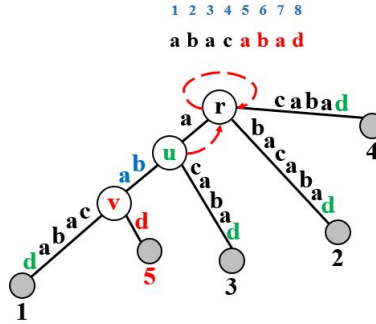
Phase: $i + 1 = 8$, **extension:** $j = 1 - 4$, $AN = (\textcircled{u})$, $rem = ba$, $last_j = 4$, $GP = 7$

As this is the start of phase 8 we first increment the global pointer so that $GP = 8$. This implicitly performs extensions $j = 1 - 4$ as shown below.



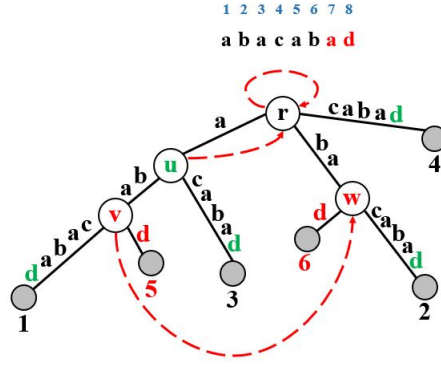
Phase: $i + 1 = 8$, **extension:** $j = 5$, $AN = (\textcircled{u})$, $rem = ba$, $last_j = 4$, $GP = 8$

This is the beginning of explicit extensions as $j = last_j + 1 = 5$ and we first skipcount down to the extension point indicated by the active node and remainder. As $AN = (\textcircled{u})$ and $rem = ba$ we make the extension directly below the a on the edge between $(\textcircled{u})$ and leaf 1. The character immediately following the extension point is $c \neq \text{str}[8] = d$ and so the extension is made via rule 2, creating a new node $(\textcircled{v})$ and leaf numbered 5. Following the extension $last_j$ is updated to $last_j = 5$ and we follow the suffix link out of the active node $(\textcircled{u})$ which leads to the root. The root becomes the new active node ($AN = (\textcircled{r})$) and note that we **do not** reduce the remainder manually here as the path has been reduced from $\text{str}[5 \dots 7] = aba$ to $\text{str}[6 \dots 7] = ba$ implicitly by following the suffix link out of $(\textcircled{u})$.



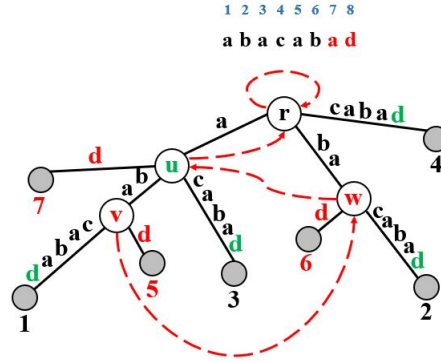
Phase: $i + 1 = 8$, **extension:** $j = 6$, $AN = (\textcircled{r})$, $rem = ba$, $last_j = 5$, $GP = 8$

We first skipcount down to the extension point indicated by the active node and remainder. As $AN = (\textcircled{r})$ and $rem = ba$ we make the extension directly below the a on the edge between $(\textcircled{r})$ and leaf 2. The character immediately following the extension point is $c \neq \text{str}[8] = d$ and so the extension is made via rule 2, creating a new node $(\textcircled{w})$ and leaf numbered 6. Following the extension $last_j$ is updated to $last_j = 6$ and as the previous extension created a new internal node $(\textcircled{v})$ we have an unresolved suffix link that should be pointed to the newly created node $(\textcircled{w})$. Finally, we follow the suffix link out of the active node $(\textcircled{r})$ which leads to the root and we reduce the remainder manually to $rem = a$.



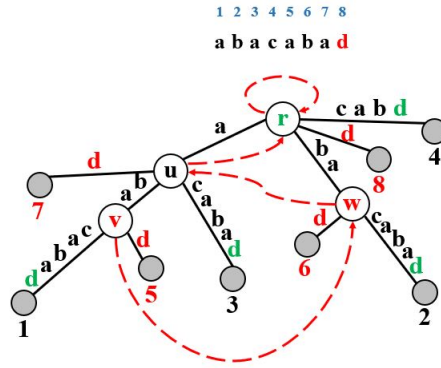
Phase: $i + 1 = 8$, **extension:** $j = 7$, $AN = \textcircled{r}$, $rem = a$, $last_j = 6$, $GP = 8$

We first skipcount down to the extension point indicated by the active node and remainder. As $AN = \textcircled{r}$ and $rem = a$ we skip over the edge between $\textcircled{r}$ and $\textcircled{u}$ arriving at $\textcircled{u}$ which becomes the active node. Similarly, having exhausted the remainder substring we set $rem = ''$ and the extension is made directly from $\textcircled{u}$. No edge starting at $\textcircled{u}$ begins with the new character $str[8] = d$ and so the extension is made via rule 2 and a new leaf node numbered 7 is created. Following the extension $last_j$ is updated to $last_j = 7$ and as the previous extension created a new internal node $\textcircled{w}$ we have an unresolved suffix link that should be pointed to $\textcircled{u}$. Finally, we follow the suffix link out of the active node ($\textcircled{u}$) which leads to the root, which becomes our new active node.



Phase: $i + 1 = 8$, **extension:** $j = 8$, $AN = \textcircled{r}$, $rem = ''$, $last_j = 7$, $GP = 8$

We first skipcount down to the extension point indicated by the active node and remainder. As $AN = \textcircled{r}$ and $rem = ''$ the extension is made directly from the root. No edge starting at $\textcircled{r}$ begins with the new character $str[8] = d$ and so the extension is made via rule 2 and a new leaf node numbered 8 is created. Following the extension $last_j$ is updated to $last_j = 8$ and we follow the suffix link out of the active node ($\textcircled{r}$) which leads to the root and we reduce the remainder manually to $rem = ''$.

**Question 6: The final phase**

Construct the regular suffix tree for $\mathbf{str}[1 \dots 8] = \text{abacabad}$ by performing one final phase to insert the character $\mathbf{str}[9] = \$$. Track the evolution of the relevant variables as we have done above.

As a final note we emphasise that although separating the final phase can be conceptually helpful, when implementing Ukkonen's algorithm to construct suffix trees we would initially append $\$$ to the string $\mathbf{str}[1 \dots n]$ and then simply perform $n + 1$ phases of the algorithm. Therefore the final phase that constructs the suffix tree for $\mathbf{str}[1 \dots n]$ from $\mathcal{I}_n$ is practically no different from any other phase of the algorithm.